

DISTANCE TRANSFORM & WATERSHED SEGMENTATION

Nabila Puspita Firdi, S.T., M.Sc.



Teknik Biomedik

Fakultas Teknologi Elektro dan Informatika Cerdas

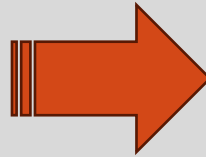
Institut Teknologi Sepuluh Nopember



OUTLINE

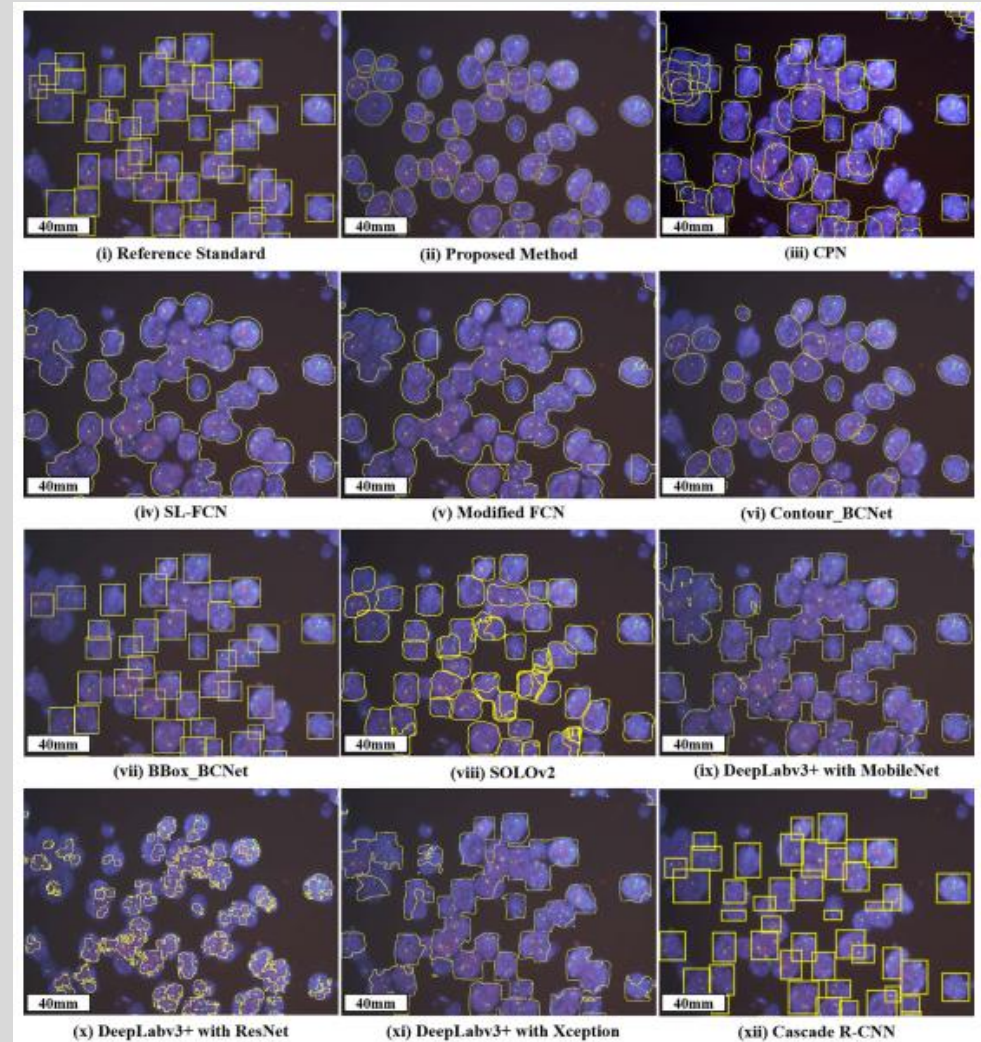
THIS WEEK (WEEK#11):

- Corner Detection:
 - Harris Corner Detection
- Line Detection and Circle Detection
 - Hough Lines Transform
 - Hough Transform for Circle Detection



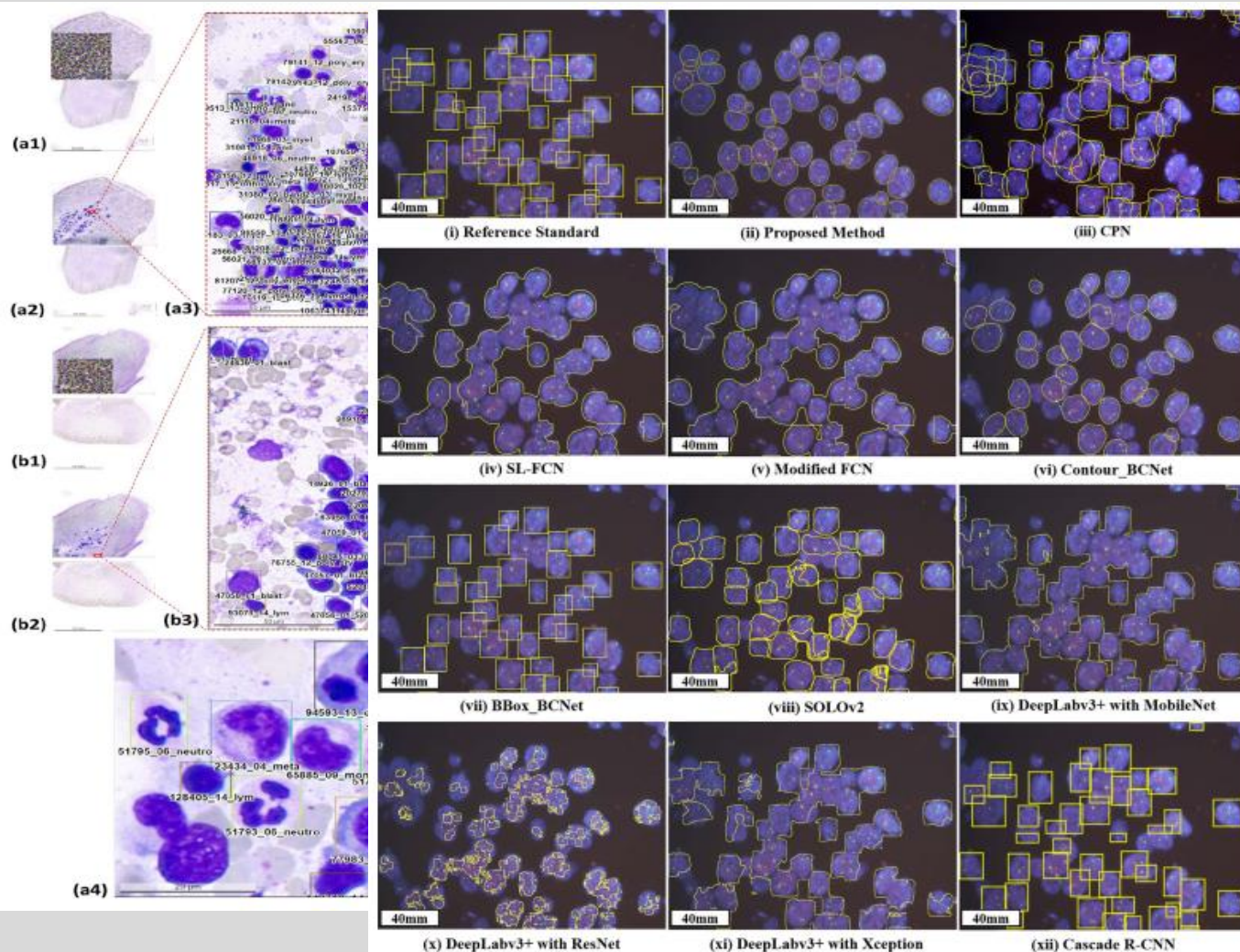
THIS WEEK (WEEK#12):

- Distance Transform
- Watershed Segmentation



Wang, C. W., Chu, K. L., Su, T. S., Liu, K. W., Lin, Y. J., & Chao, T. K. (2024). Automated quantification of HER2 amplification levels using deep learning. *IEEE Journal of Biomedical and Health Informatics*.

MEDICAL IMAGE SEGMENTATION



- Distance Transform



shape analysis and marker preparation in watershed segmentation

- Watershed Segmentation



Diferentiating overlapping objects



DISTANCE TRANSFORM



DISTANCE TRANSFORM

- Distance Transform (DT) converts a binary image into an image where each foreground pixel holds the minimum distance to the nearest background pixel.

- Given a binary image $I(u, v) = I(u)$, we denote:

$$FG(I) = \{u \mid I(u) = 1\}$$

$$BG(I) = \{u \mid I(u) = 0\}$$

- The distance transform is :

$$D(u) := \min_{u' \in FG(I)} dist(u, u')$$

- Where $u = 0, \dots, M - 1, v = 0, \dots, N - 1$ for image size $M \times N$
- $dist(u, u')$ measures the geometric distance between the two coordinate points $u = (u, v)$ and $u' = (u', v')$

DISTANCE TRANSFORM

- Euclidean distance (L_2 norm):

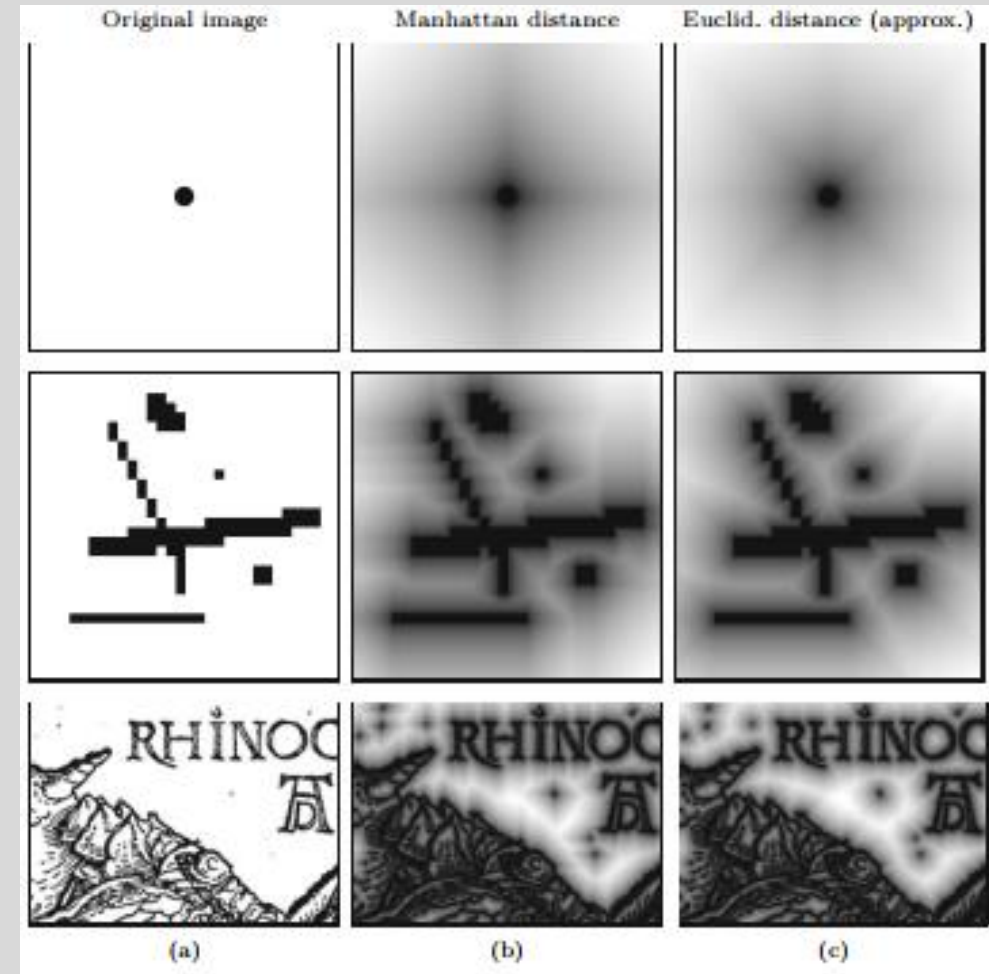
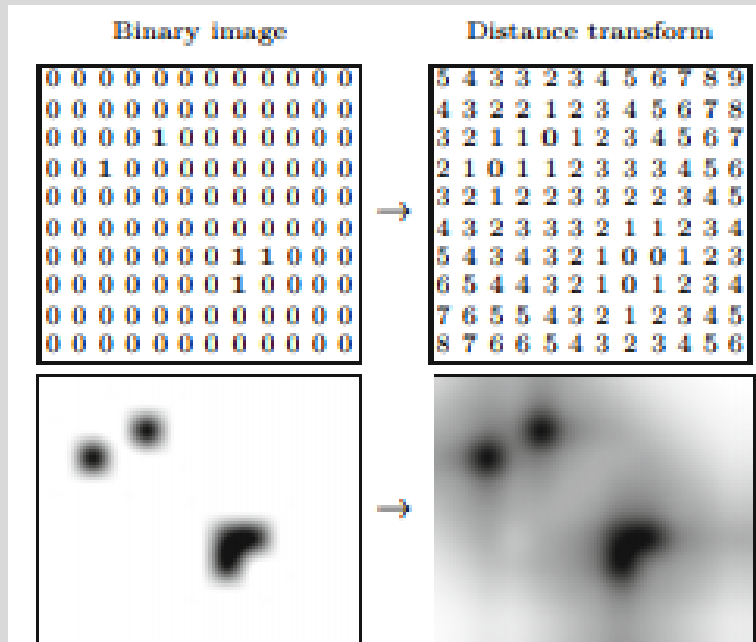
$$d_E(u, u') = \|u - u'\| = \sqrt{(u - u')^2 + (v - v')^2} \in R^+$$

- Manhattan distance (L_1 norm) :

$$d_M(u, u') = |u - u'| + |v - v'|$$

- Chessboard

$$d_C(u, u') = \max(|u - u'|, |v - v'|)$$



EUCLIDEAN DISTANCE TRANSFORM

$$d_E(u, u') = \|u - u'\| = \sqrt{|(u - u')|^2 + |(v - v')|^2} \in R^+$$

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
1	1	0	0	0
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0
0	1	1	1	0

0.00	0.00	1.00	2.00	3.00
0.00	0.00	1.00	2.00	3.00
1.00	1.00	1.41	2.00	2.24
1.41	1.00	1.00	1.00	1.41
1.00	0.00	0.00	0.00	1.00

Untuk koordinat (1,2), pixel fg terdekat ada pada koordinat (1,1)
 $(1,2) = \sqrt{|(1-1)|^2 + |(1-2)|^2} = 1$

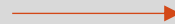
Untuk koordinat (2,2), pixel fg terdekat ada pada koordinat (1,1)
 $(2,2) = \sqrt{|(1-2)|^2 + |(1-2)|^2} = 1.41$

Untuk koordinat (2,4), pixel fg terdekat ada pada koordinat (4,3)
 $(2,4) = \sqrt{|(4-2)|^2 + |(3-4)|^2} = 2.24$

MANHATTAN DISTANCE TRANSFORM

$$d_M(u, u') = |u - u'| + |v - v'|$$

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)
1	1	0	0	0
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0
0	1	1	1	0



inisialisasi				
0	0	99	99	99
0	0	99	99	99
99	99	99	99	99
99	99	99	99	99
99	0	0	0	99

MANHATTAN DISTANCE TRANSFORM

$$d_M(u, u') = |u - u'| + |v - v'|$$

(0,0) (0,1) (0,2) (0,3) (0,4)

1	1	0	0	0
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0
0	1	1	1	0

inisialisasi

0	0	99	99	99
0	0	99	99	99
99	99	99	99	99
99	99	99	99	99
99	0	0	0	99

Forward pass

0	0	1	2	3
0	0	1	2	3
1	1	2	3	4
2	2	3	4	5
3	0	0	0	1

Untuk koordinat (2,0):

- Ori distance pada (2,0) = 99
- Atas distance pada (1,0) = 0 $\rightarrow 0+1=1$
- Kiri distance $\rightarrow$ none
- Jadi distance akhir = $\min(99,1) = 1$

Untuk koordinat (2,1):

- Ori distance pada (2,1) = 99
- Atas distance pada (1,1) = 0 $\rightarrow 0+1=1$
- Kiri distance pada (2,0) = 1 $\rightarrow 1+1=2$
- Jadi distance akhir = $\min(99,1,2) = 1$

MANHATTAN DISTANCE TRANSFORM

$$d_M(u, u') = |u - u'| + |v - v'|$$

(0,0) (0,1) (0,2) (0,3) (0,4)

1	1	0	0	0
1	1	0	0	0
0	0	0	0	0
0	0	0	0	0
0	1	1	1	0

inisialisasi

0	0	99	99	99
0	0	99	99	99
99	99	99	99	99
99	99	99	99	99
99	0	0	0	99

Forward pass

0	0	1	2	3
0	0	1	2	3
1	1	2	3	4
2	2	3	4	5
3	0	0	0	1

Untuk koordinat (3,0):

- Ori distance pada (3,0) = 99
- Atas distance pada (2,0) = 1 → 1+1=2
- Kiri distance → none
- Jadi distance akhir = min(99,2) = 2

Untuk koordinat (3,1):

- Ori distance pada (3,1) = 99
- Atas distance pada (2,1) = 1 → 1+1=2
- Kiri distance pada (3,0) = 2 → 2+1=3
- Jadi distance akhir = min(99,2,3) = 2

MANHATTAN DISTANCE TRANSFORM

$$d_M(u, u') = |u - u'| + |v - v'|$$

Forward pass

0	0	1	2	3
0	0	1	2	3
1	1	2	3	4
2	2	3	4	5
3	0	0	0	1

Backward pass

0	0	1	2	3
0	0	1	2	3
1	1	2	3	3
2	2	1	1	2
3	0	0	0	1

Untuk koordinat (4,4):

- Ori distance pada (4,4) = 1
- Kanan distance → none
- Bawah distance → none
- Jadi distance akhir = 1

Untuk koordinat (3,4):

- Ori distance pada (3,4) = 5
- Bawah distance (4,4) = 1 → 1+1=2
- Kanan distance → none
- Jadi distance akhir = min(5,2) = 2

Untuk koordinat (3,3):

- Ori distance pada (3,3) = 4
- Bawah distance (4,3) = 0 → 0+1=1
- Kanan distance (3,4)=2+1→ 3
- Jadi distance akhir = min(4,3,1) = 1

DT ALGORITHM COMPARISON

Metode	Cara kerja utama	Hitung koordinat eksplisit?
Manhattan (L_1)	Perbandingan nilai lokal +1 dari tetangga	✗ Tidak perlu
Chessboard (L_∞)	Sama dengan Manhattan tapi $\max(x, y)$	✗ Tidak perlu
Euclidean (L_2)	Hitung jarak dari koordinat foreground-background	✓ Ya, perlu

DISTANCE TRANSFORM MANHATTAN ALGORITHM

1. Initializtion:

- If the pixel is background (0), then distance = 0
- If the pixel is object (1), then the distance = ∞

2. Forward pass

- Iterate from top-left to bottom-right

$$\begin{aligned} D(x, y) \\ = \min(D(x, y), D(x - 1, y) + 1, D(x, y - 1) + 1, D(x - 1, y - 1) \\ + \sqrt{2}, D(x + 1, y - 1) + \sqrt{2}) \end{aligned}$$

3. Backward Pass

- Iterate from bottom-right to top-left

$$\begin{aligned} D(x, y) \\ = \min(D(x, y), D(x + 1, y) + 1, D(x, y + 1) + 1, D(x + 1, y + 1) \\ + \sqrt{2}, D(x - 1, y + 1) + \sqrt{2}) \end{aligned}$$

CODING TIME

```
for object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
selection at the end -add  
ob.select= 1  
if ob.select=1  
scene.objects[ob.name]  
"selected" + str(ob.name)  
ob.select = 0  
by context.select  
data.objects[one.name].select
```

```
print("please select exactly
```

```
-- OPERATOR CLASSES --
```

```
types.Operator):  
X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```


HOW TO CODE MANHATTAN DISTANCE TRANSFORM

- Import library
- Import image RGB → gray image
- Do thresholding → binary image
- Find the minimum distance
 - Forward pass
 - Backward pass

MANHATTAN DISTANCE TRANSFORM

```
def distance_transform(binary):  
    h, w = binary.shape  
    inf = h + w # maximum possible distance  
    dist = np.full((h, w), inf, dtype=np.int32)  
    dist[binary == 0] = 0 # background = 0 distance  
  
    # Forward scan  
    for y in range(h):  
        for x in range(w):  
            if dist[y, x] != 0:  
                top = dist[y - 1, x] if y > 0 else inf  
                left = dist[y, x - 1] if x > 0 else inf  
                dist[y, x] = min(dist[y, x], top + 1, left + 1)  
  
    # Backward scan  
    for y in range(h - 1, -1, -1):  
        for x in range(w - 1, -1, -1):  
            if dist[y, x] != 0:  
                bottom = dist[y + 1, x] if y < h - 1 else inf  
                right = dist[y, x + 1] if x < w - 1 else inf  
                dist[y, x] = min(dist[y, x], bottom + 1, right + 1)  
  
    return dist
```

Initiation

Forward scan
calculation

Backward scan
calculation

MANHATTAN DISTANCE TRANSFORM

```
# Forward scan
for y in range(h):
    for x in range(w):
        if dist[y, x] != 0:
            top = dist[y - 1, x] if y > 0 else inf
            left = dist[y, x - 1] if x > 0 else inf
            dist[y, x] = min(dist[y, x], top + 1, left + 1)
```

Forward pass : Iterate from top-left to bottom-right

$$D(x, y) = \min(D(x, y), D(x - 1, y) + 1, D(x, y - 1) + 1, D(x - 1, y - 1) + \sqrt{2}, D(x + 1, y - 1) + \sqrt{2})$$

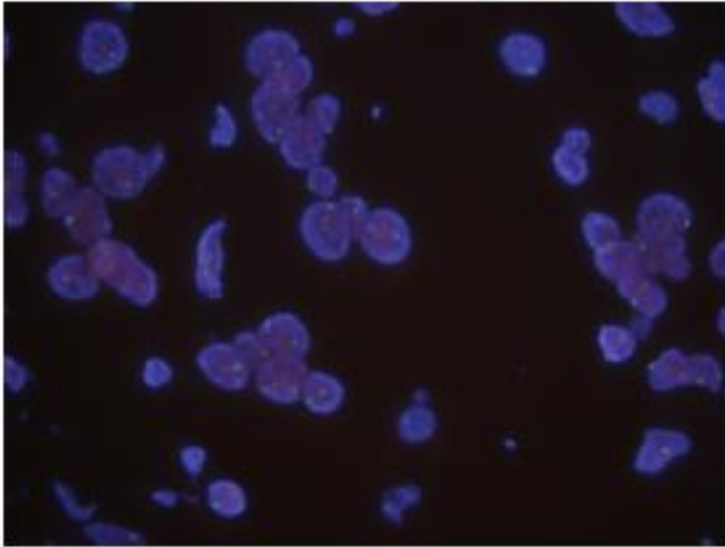
```
# Backward scan
for y in range(h - 1, -1, -1):
    for x in range(w - 1, -1, -1):
        if dist[y, x] != 0:
            bottom = dist[y + 1, x] if y < h - 1 else inf
            right = dist[y, x + 1] if x < w - 1 else inf
            dist[y, x] = min(dist[y, x], bottom + 1, right + 1)
```

Backward pass: Iterate from bottom-right to top-left

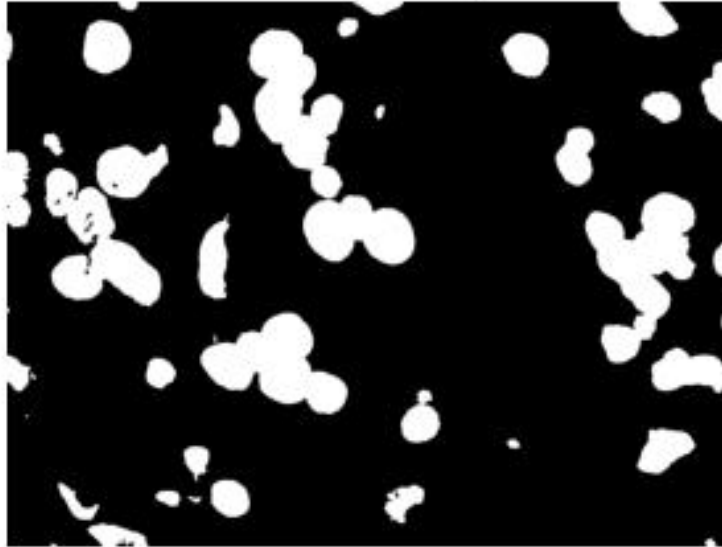
$$D(x, y) = \min(D(x, y), D(x + 1, y) + 1, D(x, y + 1) + 1, D(x + 1, y + 1) + \sqrt{2}, D(x - 1, y + 1) + \sqrt{2})$$

MANHATTAN DISTANCE TRANSFORM

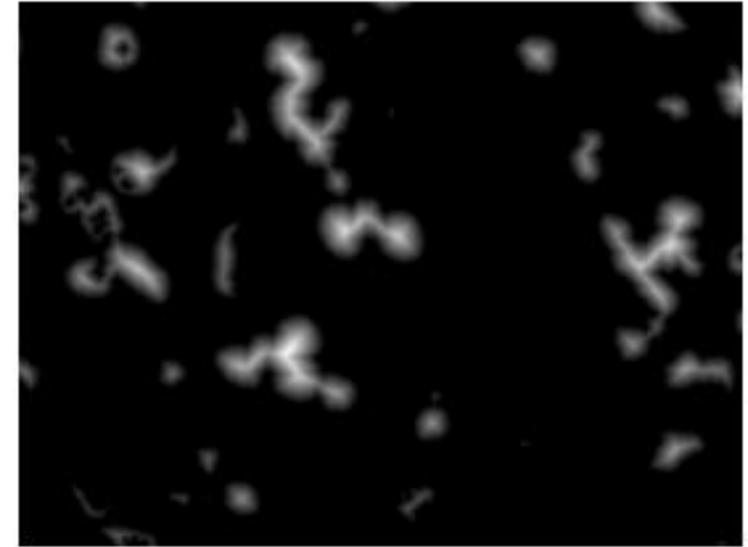
Original Image



Binary Image



Distance Transform



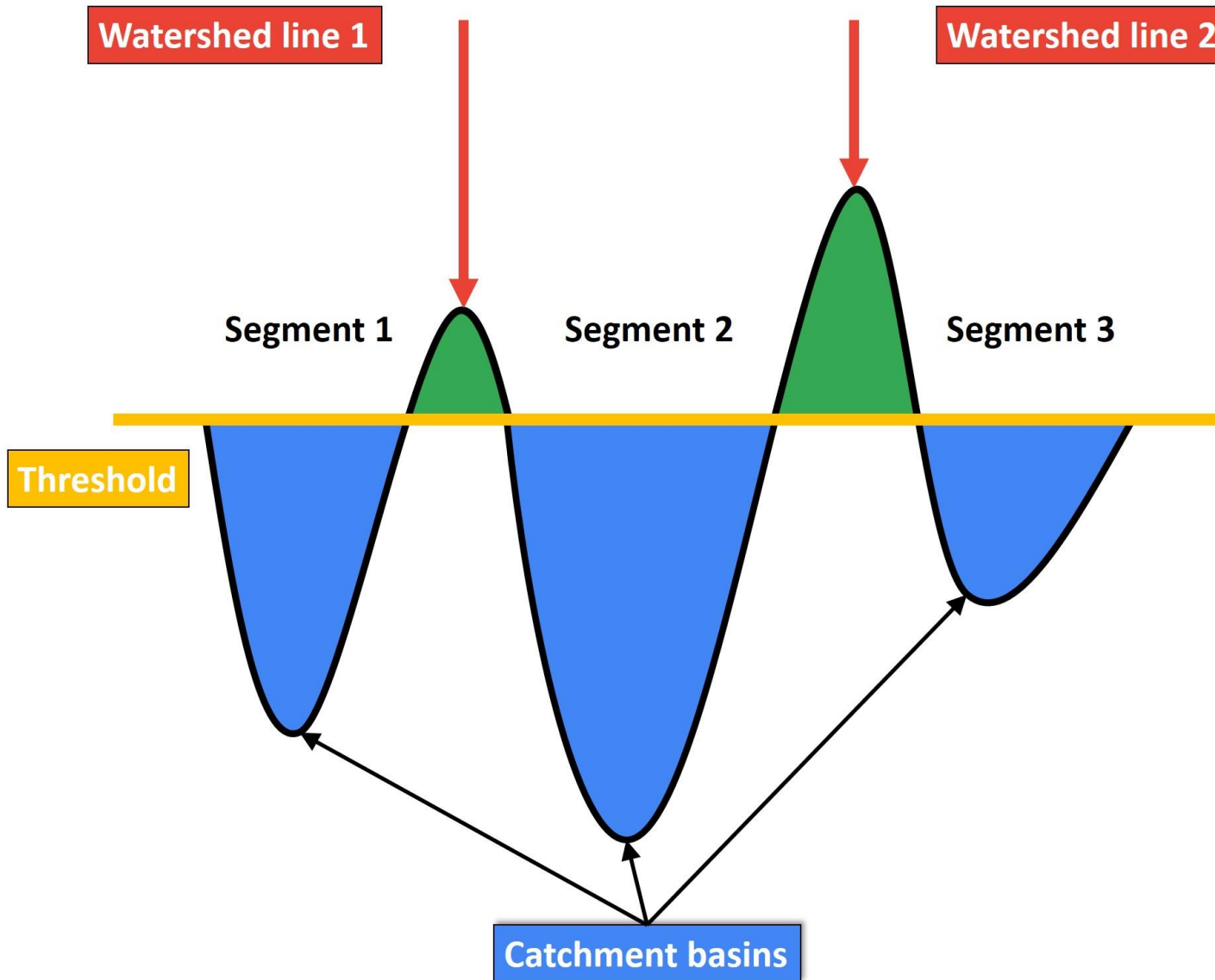


WATERSHED SEGMENTATION



WATERSHED SEGMENTATION

- Watershed algorithm treats the grayscale image as a topographic surface:
 - Low intensity = valleys
 - High intensity = peaks
- When watershed is useful?
 - If objects are overlapping and simple thresholding might be failed
 - When an image have a very soft contrast within objects
 - Object segmentation, not only separating foreground and background
- Medical example:
 - Blood cells segmentation (RBC, WBC)
 - Separating overlapping cell nuclei
 - Detecting tumor boundaries



WATERSHED SEGMENTATION

- How the algorithm works:
 - Imagine pouring water to deepest valley (darkest point in image)
 - The water will spread around it (neighborhood pixels)
 - When the water from two places nearly meet, create a separator line → boundary between objects.

WATERSHED SEGMENTATION

- Main idea: Separating the image area Ω into several parts $R_1, R_2, \dots, R_n$ with each part:
 - Have one starting point (darkest pixel)
 - All pixels within it point to that starting point
 - Not overlap

$$\Omega = \bigcup_{i=1}^n R_i \text{ with } R_i \cap R_j = \emptyset \text{ if } i \neq j$$

- One object R_i contains all pixels that naturally “flow” towards the darkest point, then:
 $R_i = \{(x, y) \in \Omega \mid \text{path to } l_i \text{ has decreasing intensity from } g(x, y)\}$

CODING TIME

```
for object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
ob.select= 1  
if ob.select=1  
    scene.objects[ob.name]  
    ["selected"] + str(ob.name)  
    mirror_mod.select = 0  
    scene.objects[ob.name].select
```

```
print("please select exactly  
-- OPERATOR CLASSES --
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```


HOW TO CODE WATERSHED SEGMENTATION

- Import library
- Import image RGB → gray image
- Do thresholding → binary image
- Calculate the distance transform
- Find the local minima
- Labeling for each local minima → as an object marker
- Apply watershed segmentation
- Find the boundary lines between objects from the watershed results and thicken the lines.

WATERSHED SEGMENTATION

```
# Convert image to grayscale if it's not already
if len(image.shape) == 3:
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
else:
    gray_image = image
```

RGB to gray

```
otsu_threshold, _ = cv2.threshold(gray_image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
binary_img = gray_image > otsu_threshold
binary_img = binary_img.astype(np.uint8)
```

Gray to binary

```
binary_dil_img = binary_dilation(binary_img, disk(2))
binary_dil_img = binary_dil_img.astype(np.uint8)
```

```
##distance transform
# dist_result = distance_transform(binary_dil_img)
dist_result = cv2.distanceTransform(binary_dil_img, cv2.DIST_L2, 3)
minima = h_minima(-dist_result, h=1)
```

Distance transform &
looking for local minima

```
markers = label(minima)
labels_ws = watershed(-dist_result, markers, mask=binary_dil_img)
```

Labeling, and do
watershed segmentatin

```
boundaries = find_boundaries(labels_ws, mode='inner', background=0)
thick_boundaries = binary_dilation(boundaries, disk(1))
detected_obj = image.copy()
detected_obj[thick_boundaries] = [255, 0, 0]
```

Find the boundary lines between
objects from the watershed results and
thicken the lines.

WATERSHED SEGMENTATION

```
##distance transform
# dist_result = distance_transform(binary_dil_img)
dist_result = cv2.distanceTransform(binary_dil_img, cv2.DIST_L2, 3)
minima = h_minima(-dist_result, h=1)
```

```
markers = label(minima)
labels_ws = watershed(-dist_result, markers, mask=binary_dil_img)
```

```
skimage.measure.label(label_image, background=None, return_num=False,
connectivity=None) \[source\]
```

<https://scikit-image.org/docs/stable/api/skimage.measure.html#skimage.measure.label>

Find all connected foreground objects, assigned unique label to each connected region

```
skimage.segmentation.watershed(image, markers=None, connectivity=1,
offset=None, mask=None, compactness=0, watershed_line=False) \[source\]
```

<https://scikit-image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.watershed>

Find watershed basins in an image;
Markers: the desired number of basins
Mask: only point at which mask == True will be labeled

```
void cv::distanceTransform ( InputArray  src,
                           OutputArray dst,
                           OutputArray labels,
                           int          distanceType,
                           int          maskSize,
                           int          labelType = DIST_LABEL_CCOMP
                           )
```

https://docs.opencv.org/3.4/d7/d1b/group_imgproc_misc.html#ga8a0b7dfcb7a13dde018988ba3a43042

- DIST_L1 → Manhattan distance
- DIST_L2 → Euclidean distance
- DIST_C → Chessboard distance

```
skimage.morphology.h_maxima(image, h, footprint=None)
```

https://scikit-image.org/docs/stable/api/skimage.morphology.html#skimage.morphology.h_maxima

Determine all minima of the image with depth $\geq h$;
 h : minimal depth of all expected minima (unsigned integer)

WATERSHED SEGMENTATION

```
boundaries = find_boundaries(labels ws, mode='inner', background=0)
thick_boundaries = binary_dilation(boundaries, disk(1))
detected_obj = image.copy()
detected_obj[thick_boundaries] = [255, 0, 0]
```

```
skimage.segmentation.find_boundaries(label_img, connectivity=1,  
mode='thick', background=0) \[source\]
```

[https://scikit-](https://scikit-image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.find_boundaries)

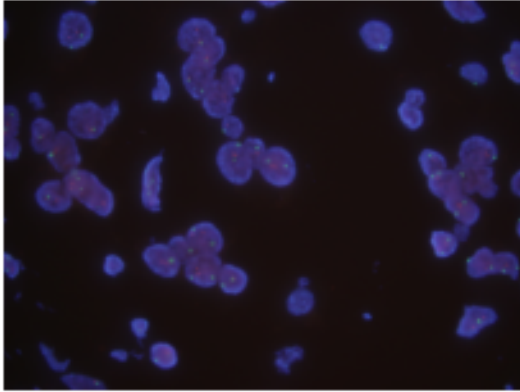
[image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.find_boundaries](https://scikit-image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.find_boundaries)

Find the boundaries;

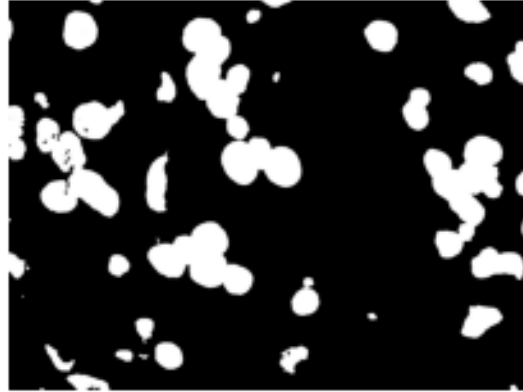
Mode 'inner': outline the pixels just inside of objects, leaving background untouched

WATERSHED SEGMENTATION RESULTS

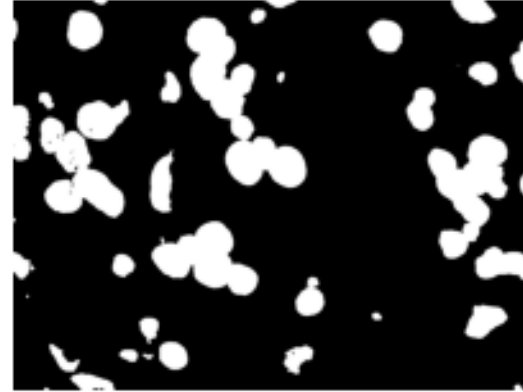
Original Image



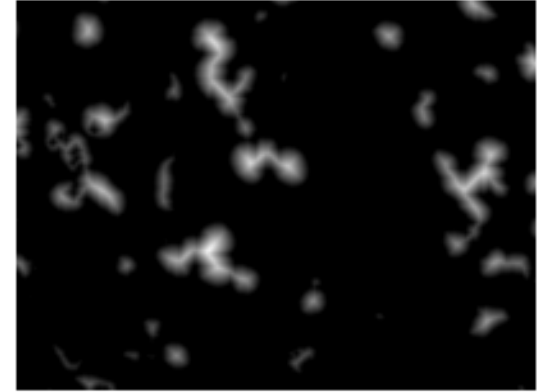
Binary Image



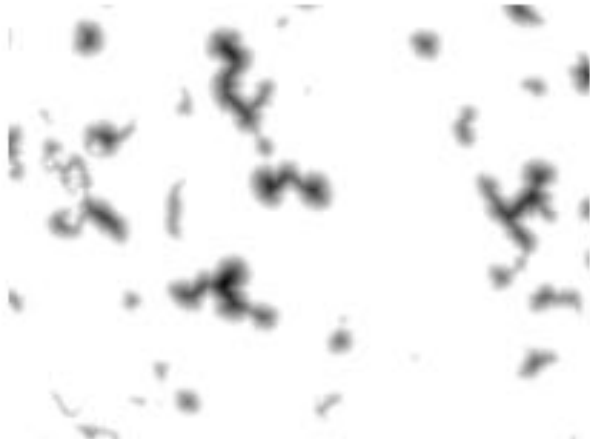
Binary dilation



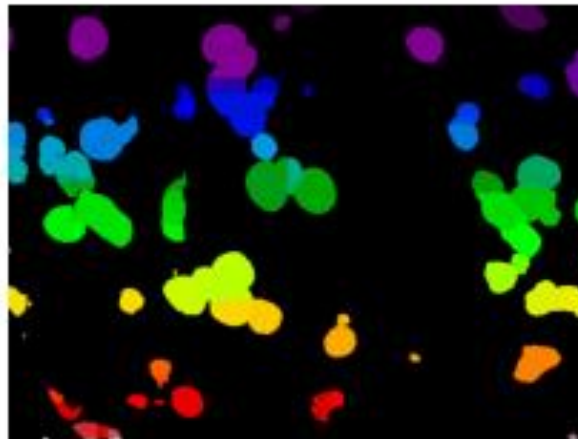
Distance Transform



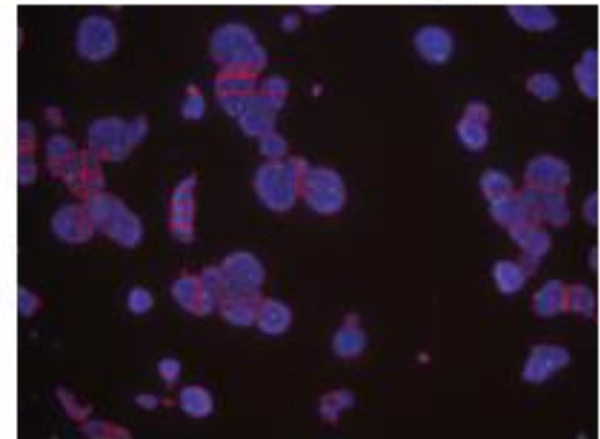
dist minima



watershed

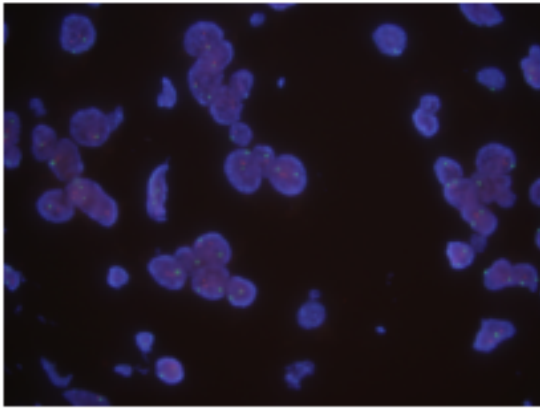


Detected obj

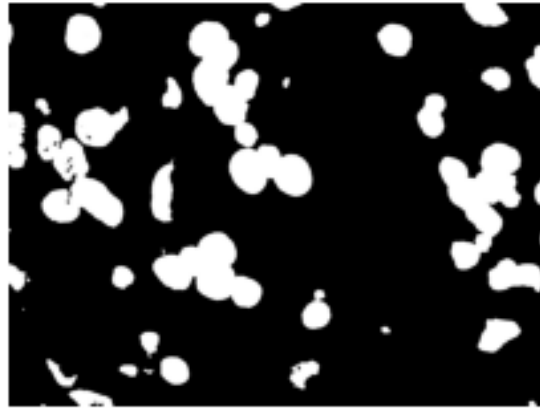


WATERSHED SEGMENTATION RESULTS

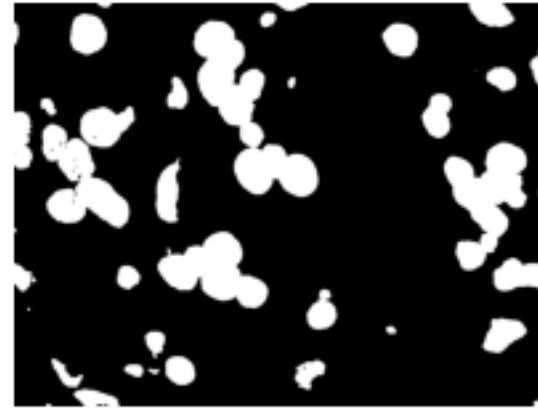
Original Image



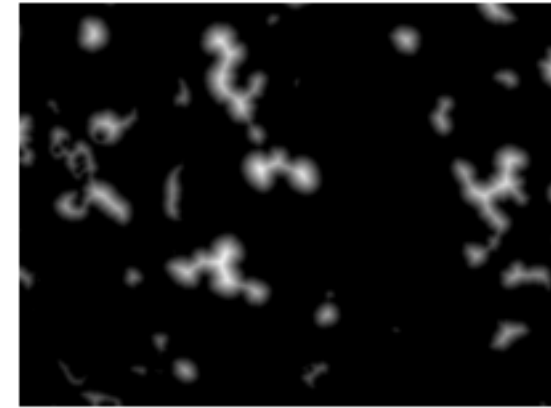
Binary Image



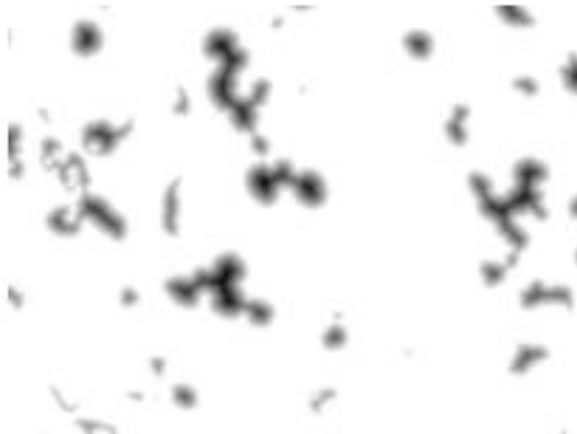
Binary dilation



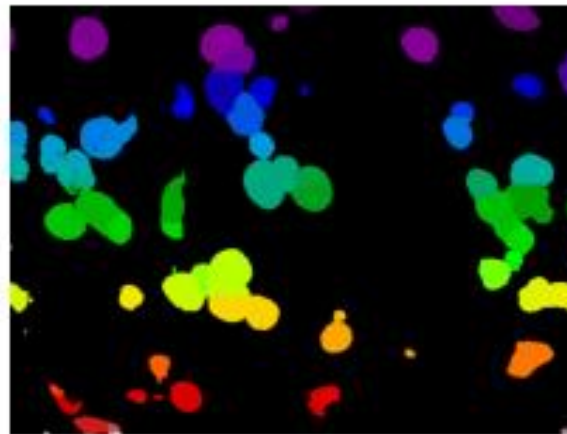
Distance Transform



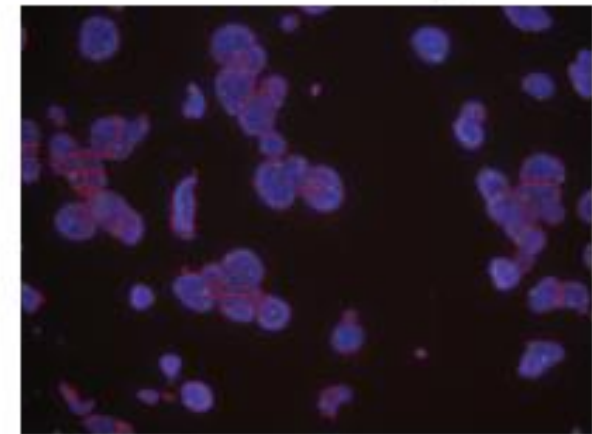
dist minima



watershed

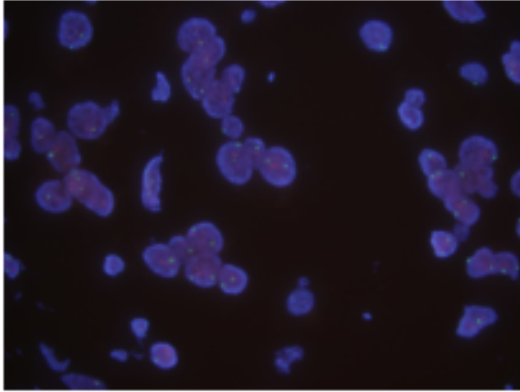


Detected obj

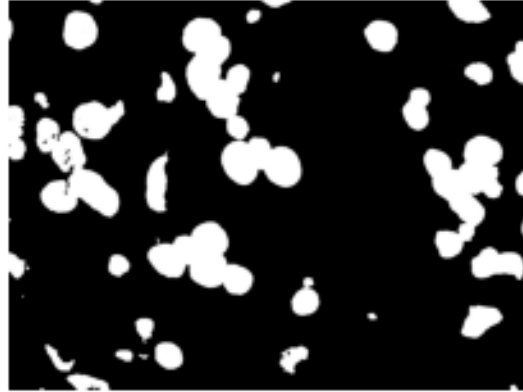


WATERSHED SEGMENTATION RESULTS

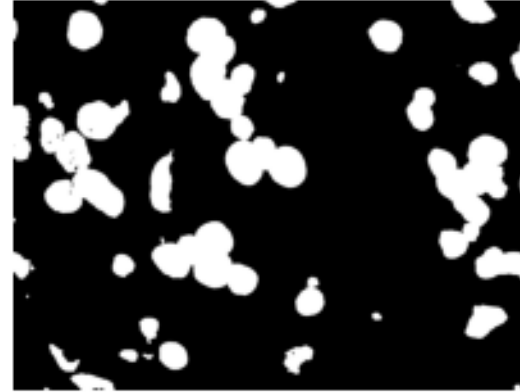
Original Image



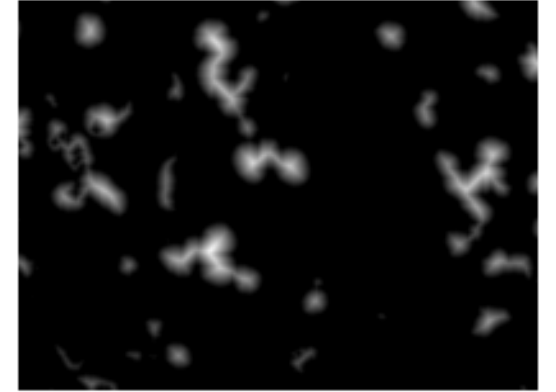
Binary Image



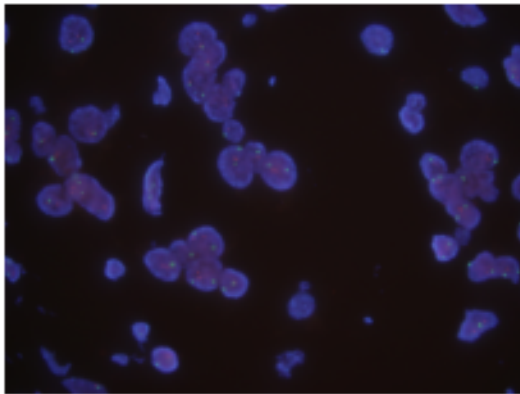
Binary dilation



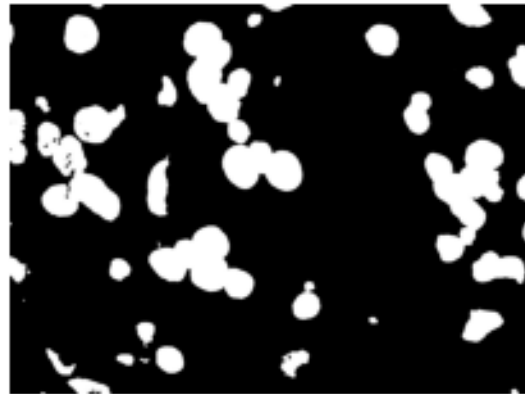
Distance Transform



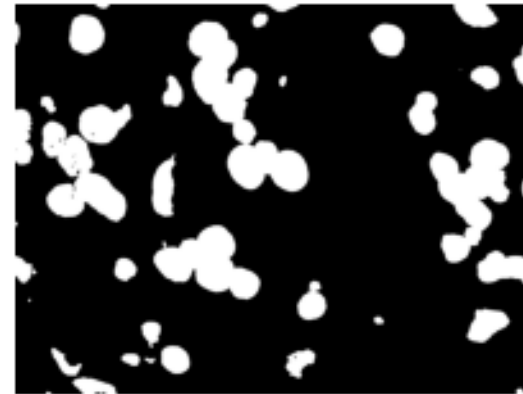
Original Image



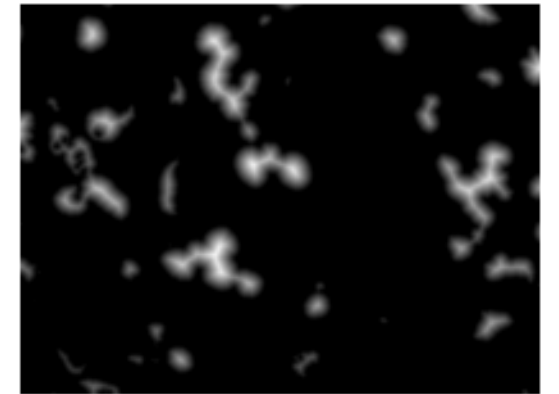
Binary Image



Binary dilation

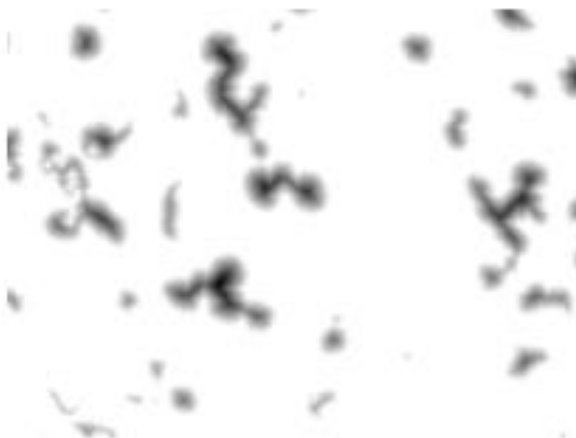


Distance Transform

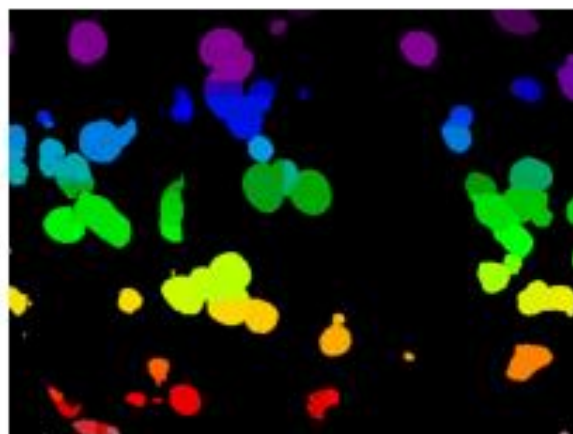


WATERSHED SEGMENTATION RESULTS

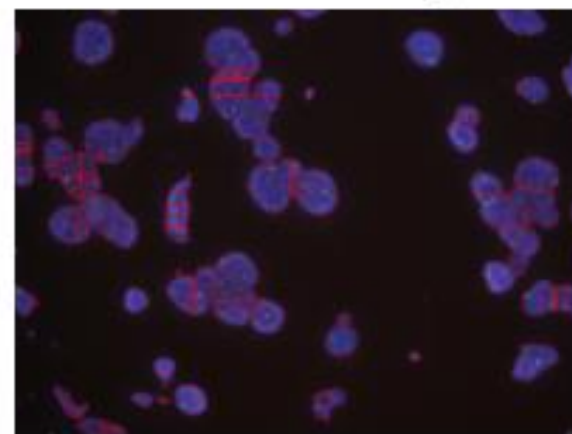
dist minima



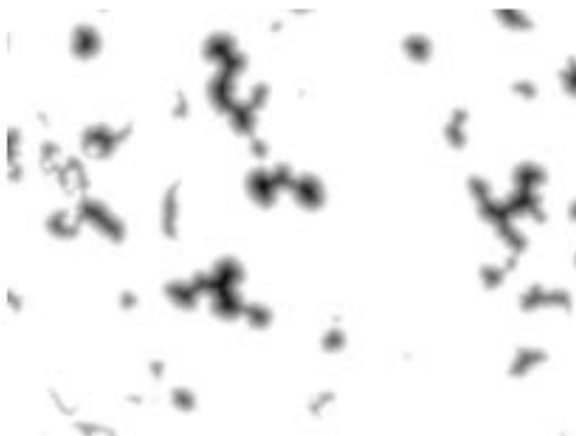
watershed



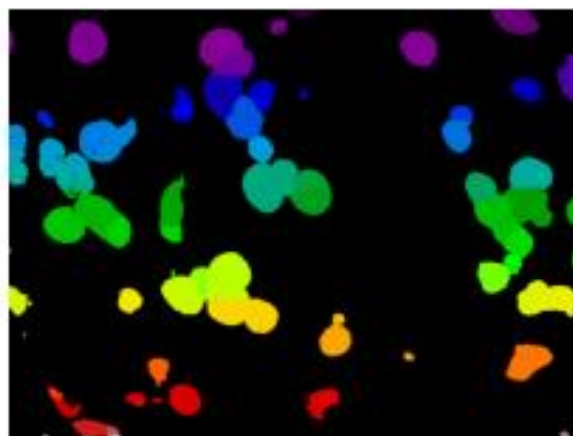
Detected obj



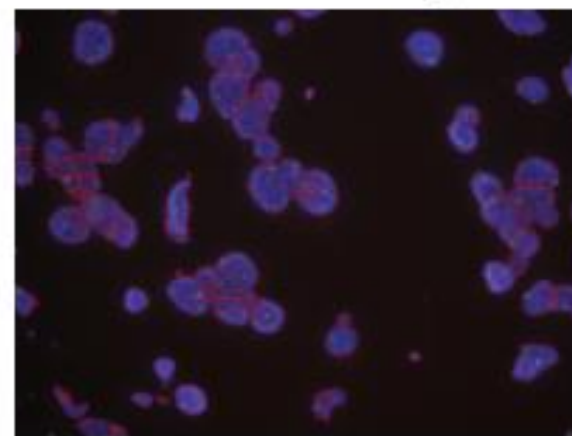
dist minima



watershed



Detected obj



REFERENCE

- Burger, W., & Burge, M. J. (2022). Digital image processing: An algorithmic introduction. Springer Nature
- Wang, C. W., Huang, S. C., Khalil, M. A., Hong, D. Z., Meng, S. I., & Lee, Y. C. (2023). CW-NET for multitype cell detection and classification in bone marrow examination and mitotic figure examination. Bioinformatics, 39(6), btad344.
- Wang, C. W., Chu, K. L., Su, T. S., Liu, K. W., Lin, Y. J., & Chao, T. K. (2024). Automated quantification of HER2 amplification levels using deep learning. IEEE Journal of Biomedical and Health Informatics.
- https://docs.opencv.org/3.4/d7/d1b/group_imgproc_misc.html#ga8a0b7fdfeb7a13dde018988ba3a43042
- https://scikit-image.org/docs/stable/api/skimage.morphology.html#skimage.morphology.h_maxima
- <https://scikit-image.org/docs/stable/api/skimage.measure.html#skimage.measure.label>
- <https://scikit-image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.watershed>
- https://scikit-image.org/docs/stable/api/skimage.segmentation.html#skimage.segmentation.find_boundaries

THANK YOU